

MACHINE LEARNING AND HPC

Adrian Jackson

a.jackson@epcc.ed.ac.uk



THE UNIVERSITY
of EDINBURGH



ML not AI

- Machine Learning is an implementation tool and a subset of artificial intelligence (AI) research
- AI covers:
 - ML
 - Natural Language Processing (NLP)
 - Decision and planning systems
 - Knowledge representation
 - Perception systems
- NLP uses ML tools
 - ML is both a field and tools for implementing machine intelligence approaches

NLP (LLMs) is not ML and vice versa

- NLP requires
 - Semantic analysis
 - Syntactic analysis
 - Combining both systems to process and respond to language questions
- Leverages multiple ML approaches and other systems to deliver NLP functionality
 - NLP systems are complex data processing, conversion, and optimisation processes
 - Couple together the language models with image generators, code generators, etc..., which are ML systems in their own right

Machine Learning

- Four classes of algorithms

- Supervised

- Known good and bad results
 - Learning based on training from inputs + desired output

- Unsupervised

- No predefined structure in the data
 - Classification (grouping)

- Semi-supervised

- Partially labelled datasets
 - Classify or forecast based on labelled part, with unlabelled filling out the overall data set

- Reinforcement

- Learn by experiencing/interaction with data/conditions

Example of ML

- Reconstructing radio astronomy and MRI images
 - New algorithms use optimisation theory to converge to high precision images
 - Algorithms alternate between regularisation (denoising) and image model steps (measurement operator)
 - Denoising is expensive, but can be replaced by ML
 - ML training for denoising similar to ImageNet etc... (DNN, lots of source images, weights to updated, etc...)
 - Inference (image production) requires denoiser then measurement operator step
 - Measurement operator dominated by FFT operations
 - Can port the FFT to the GPU, i.e. everything done on the GPU, but still separate stepsImage size limited by GPU memory (potential issue for a lot of “simulation” ML

Machine Learning

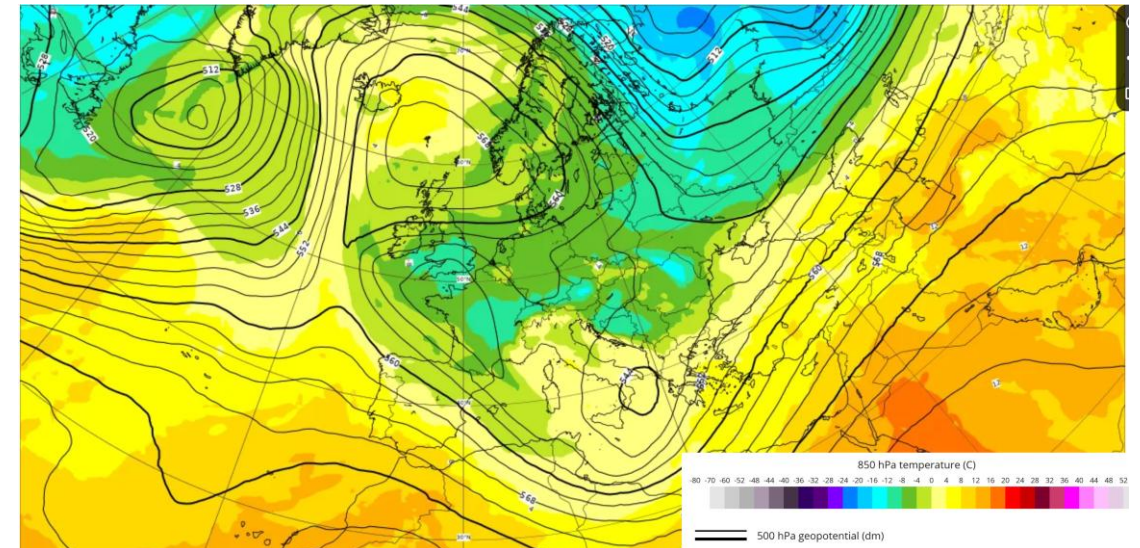
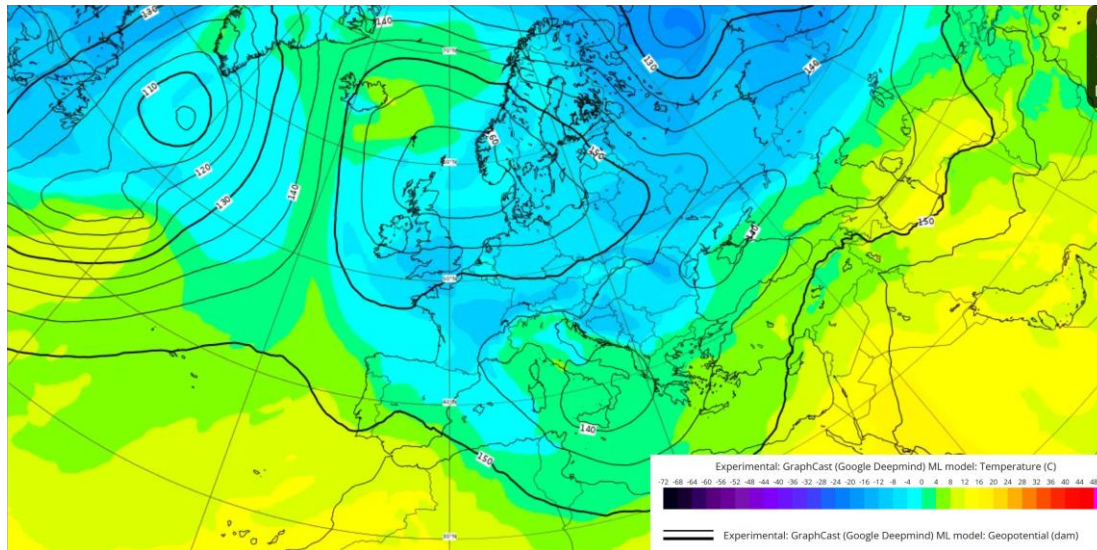
- Implementing these approaches requires some software and frameworks
 - How do you “learn” data patterns or formats or features
- Different approaches
 - Currently dominate ones are deep neural networks (DNNs)
 - Large number of parameters that can be updated automatically
 - Loss function that represents the aggregate of these and the outputs of the network
 - Update the parameters based on the optimisation of the loss function
 - Converge to minimal loss and small(ish) weights
 - Other approaches also available
 - Graph Neural Networks: The network that is built encodes some spatial locations (i.e. points on earth, place in a social network graph, etc...

Machine Learning

- Machine learning has two main phases
- Training
 - The process of tuning the network to be able to process data and produce outcomes
 - This is generally the expensive part
 - Can require significant computational resources to host the network and/or process large datasets to get a robust implementation
- Inference
 - Using a trained network to process data
 - Generally much faster than network training
 - No iterative processes, no differentiation of functions, etc...
 - Simply pass the data through the network with fixed weights, requiring a (relatively) small number of matrix-vector operations

Examples of ML

- GraphCast: Graph neural network implementation of weather forecasting functionality
 - Vertices on the graphs represent spatial data (data where the order of connections matter)



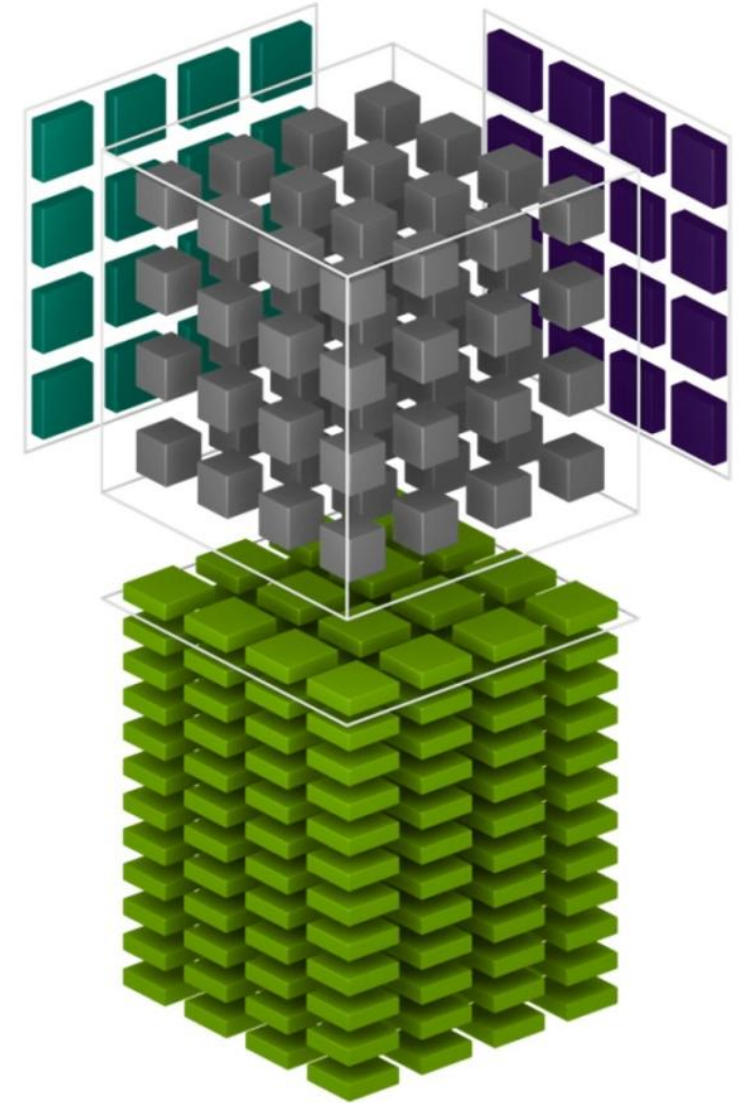
- Inference
 - Full forecast with 11 data fields in less than a minute on a single TPUv4
 - Adapted to provide discrete forecast points
 - Biggest computational challenge getting in the input data

High Performance Computing

- Hardware chosen for high performance across one or dimensions of:
 - Processors
 - Data generation/transformation
 - Memory
 - Temporary storage at high performance
 - Interconnect
 - Communication between components/nodes and the outside world
 - Storage
 - Permanent data location
- Machine learning simply another application
 - But big enough user base to enable significant specialisation/customisation of hardware

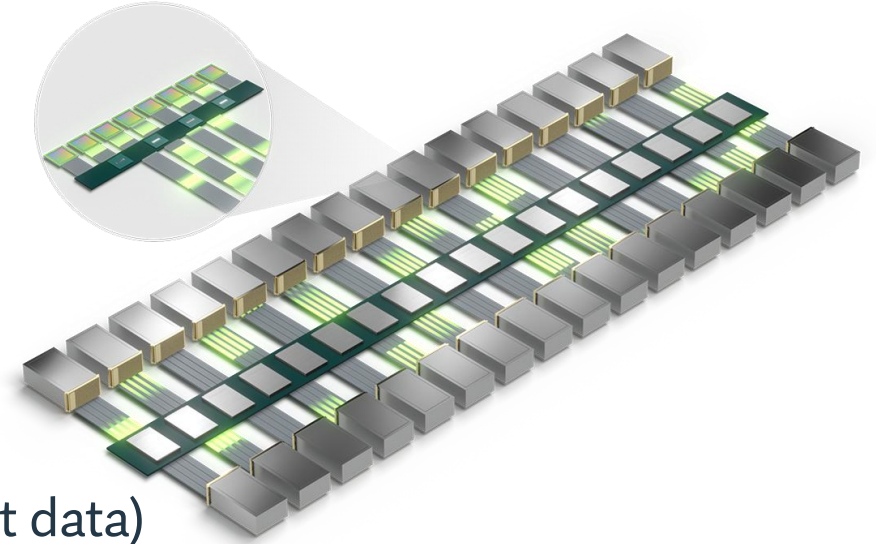
High Performance Computing

- Machine learning heavily dependent on GPUs or other custom processors
 - Focus on matrix-matrix, matrix-vector operations
 - Subset of generalised high performance computing
- Standardised operations allows efficient mapping to GPU hardware
 - SIMT approach, with lock step and over subscription
 - Local storage and coalesced memory operations
 - Customised reduction operations for matrix products
- Significant different in the precision of operations
 - 16-bit, 8-bit data structures
 - Tensor cores for specialised matrix-vector operations with defined accumulation



High Performance Computing

- Network is important for training
 - High bandwidth low latency improved training efficiency
 - Specific GPU-GPU networks (NVLink)
 - Network limit can set training domain size
 - Inference generally doesn't need it (except for sending input data)
- Storage requirements depend on ML being undertaken
 - Training the main target
 - Could be large number of relatively small files
 - i.e. images for ImageNet, corpus documents for LLM
 - Often needs some functionality to re-arrange the files for different processes (swap input data)
- Underlying software technology same as traditional parallel computing
 - blas/lapack maths libraries
 - MPI for network communications
 - CUDA for GPU programming (Nvidia)



Machine Learning at Scale

- Machine learning training generally requires high performance computing resources for *achievable* performance
 - GPUs, TPUs, etc...
 - High performance storage
 - etc...
- Large scale ML training requires HPC systems
 - All the above
 - Plus high performance networks
- All is possible without HPC, but negates the benefits
- Inference may also need HPC
 - Depending on the scale required
 - Large Language Model (LLM) deployment uses HPC
 - Different requirements (less close coupling)

Efficient/effective usage

- As with any application:
 - Match/fit the software to the hardware
 - Reduce bottlenecks
 - Aim for load balance
- Python + Framework can make this harder
 - Not generally writing software functionality yourself
 - Rely on getting optimised software packages configured and deployed
 - Rely on network design/parameter configuration/package choices, etc...
- Running on optimised hardware can make a big difference
 - Specific GPUs
 - High performance networks
 - etc...

Frameworks

- Most are not writing their own ML software
 - General networks require the same functionality
 - Reusable components
 - Optimisers
 - Distribution functionality
 - Automated construction
- Pre-existing network architectures commonly leveraged
 - U-Net, Res-Net: image segmentation
 - BERT, ChatGPT, etc...: language models
 - Dall-E: Generative networks
 - etc...

Multi-GPU calculations

- Large scale networks have large computational requirements
 - Multiple GPUs enable faster training and handling of larger models and datasets
 - Key for reducing training time and achieving state-of-the-art results
- Different ways to parallelise DNN training
 - Data Parallelism:
 - Training data is split across GPUs
 - Each GPU computes gradients independently
 - Gradients are averaged to update the model
 - Allows scaling to multiple GPUs with minimal code changes
 - Model Parallelism:
 - Model layers are split across GPUs
 - Each GPU processes a portion of the model
 - Requires more complex synchronization
 - Useful for very large models that don't fit in a single GPU

Multi-GPU calculations

- Benefits:
 - Faster training times and reduced time to convergence
 - Ability to work with larger models and datasets
- Challenges:
 - Increased complexity in code and debugging
 - Synchronization and communication overhead
 - Diminishing returns beyond a certain number of GPUs
- Considerations for Multi-GPU Training
 - Batch Size: Adjust the batch size per GPU to fully utilize memory
 - Data Parallelism Strategy: Choose synchronous or asynchronous gradient updates
 - Scaling Efficiency: Measure the speedup achieved with additional GPUs
 - Memory Optimization: Implement gradient checkpointing or mixed precision training for memory efficiency

Data Storage

- Interference
 - Need to be able to process and return data/output
 - Large scale parallel filesystems required
- Training
 - Depends on the type of data used for training
 - Lots of small amounts of files and data
 - Need to share between worker processes
 - Data loading and processing can be a significant bottleneck
- Potential benefits from
 - Node local storage
 - Object storage

- Questions?
- a.jackson@epcc.ed.ac.uk

